

docrag

Tutorial de Uso dos Agentes

Consulta e Revisao de Codigo com IA

Versao 1.0 - Junho 2026

Repositorio: github.com/tsvsampaio/docrag

. Sumario

- 1 Introducao
- 2 Pre-requisitos
- 3 Configuracao do Ambiente
- 4 Agente Expert - Consulta a Documentacao
 - 4.1 Sintaxe basica
 - 4.2 Exemplos de perguntas
 - 4.3 Entendendo a resposta
- 5 Agente Revisor - Correcao deCodigo
 - 5.1 Sintaxe basica
 - 5.2 Exemplo de revisao
 - 5.3 Ferramentas do revisor
- 6 Playground - Interface Web
 - 6.1 Iniciando o Playground
 - 6.2 Agentes disponiveis
 - 6.3 Interagindo com os agentes
- 7 Cenarios de Uso
- 8 Solucao de Problemas
- 9 Referencias

1. Introducao

O docrag e um sistema de RAG (Retrieval-Augmented Generation) que combina a documentacao do framework agno com agentes de IA para oferecer tres servicos principais:

- Agente Expert: consulta a documentacao indexada em uma base vetorial (ChromaDB) e responde perguntas sobre o framework agno em portugues ou ingles.
- Agente Revisor: analisa arquivos de codigo Python, executa linter e formatador, e retorna o codigo corrigido completo.
- Agente Tradutor: traduz documentacao Markdown do ingles para o portugues brasileiro, preservando blocos de codigo e estrutura.

Voce pode interagir com todos os agentes de duas formas:

- Pelo terminal (CLI) com comandos simples
- Pelo Playground, uma interface web interativa no navegador

Os agentes utilizam modelos da OpenAI:

- Expert: GPT-4o-mini (rapido e economico para consulta a documentacao)
- Revisor: GPT-4o (mais preciso para geracao e correcao de codigo)
- Tradutor: GPT-4o-mini (otimizado para traducao)

O embedder text-embedding-3-small e usado para indexacao vetorial.

2. Pre-requisitos

Antes de utilizar os agentes, certifique-se de ter:

- Python 3.12 ou superior
- uv (gerenciador de pacotes) ou pip
- Chave de API da OpenAI (OPENAI_API_KEY)
- Conexao com a internet (para chamadas as APIs)
- O codigo-fonte do projeto clonado do GitHub

3. Configuracao do Ambiente

3.1. Clonar o repositorio

```
$ git clone https://github.com/tsvsampaio/docrag.git
```

```
$ cd docrag
```

3.2. Criar ambiente virtual com uv

```
$ uv venv --python 3.12
```

```
$ .venv\Scripts\activate
```

```
$ uv pip install -r requirements.txt
```

3.3. Configurar variaveis de ambiente

Crie um arquivo .env na raiz do projeto com sua chave da OpenAI:

```
OPENAI_API_KEY=sk-sua-chave-aqui
```

A chave OPENAI_API_KEY é obrigatória para o funcionamento de ambos os agentes. Ela é usada tanto para o embedder (indexação) quanto para o modelo de linguagem (GPT-4o-mini).

3.4. Verificar a instalação

```
$ python cli.py --help
```

O comando acima deve exibir a lista de subcomandos disponíveis: crawl, translate, index, ask, review e playground.

Você também pode usar o entry point registrado via uv:

```
$ uv run docrag --help
```

4. Agente Expert - Consulta a Documentacao

O Agente Expert e um assistente especializado no framework agno. Ele utiliza duas bases de conhecimento indexadas no ChromaDB:

- agno_docs_en: documentacao original em ingles (1041 documentos)
- agno_docs_pt: documentacao traduzida para portugues (1041 documentos)

O agente detecta automaticamente o idioma da pergunta e responde no mesmo idioma, priorizando a base de conhecimento em portugues quando a pergunta for em PT-BR.

4.1. Sintaxe basica

```
$ python cli.py ask "sua pergunta aqui"
```

```
$ uv run docrag ask "sua pergunta aqui"
```

Ambos os comandos retornam uma resposta formatada em Markdown com explicacoes e exemplos de codigo quando relevante. Use a forma que preferir.

4.2. Exemplos de perguntas

Perguntas em portugues:

```
$ python cli.py ask "O que e agno e como criar um agente?"
```

```
$ python cli.py ask "Como usar tools em agentes agno?"
```

```
$ python cli.py ask "Qual a diferenca entre Agent e Workflow?"
```

Perguntas em ingles:

```
$ python cli.py ask "How to create a custom tool in agno?"
```

```
$ python cli.py ask "How does Knowledge work in agno 2.x?"
```

4.3. Entendendo a resposta

A resposta do Expert contem:

- Uma explicacao direta da pergunta
- Exemplos de codigo com syntax highlighting (bound por ```)
- Referencia a documentacao oficial quando aplicavel
- Links uteis para aprofundamento

O agente consulta a base vetorial em tempo real para cada pergunta, garantindo que a resposta seja baseada na documentacao mais recente do agno que foi indexada.

5. Agente Revisor - Correcao de Codigo

O Agente Revisor e um assistente especializado em revisao de codigo Python. Ele le o arquivo, analisa o codigo, executa ferramentas de qualidade (linter e formatador) e retorna o codigo corrigido completo com explicacoes das alteracoes realizadas.

5.1. Sintaxe basica

```
$ python cli.py review caminho/para/arquivo.py
```

```
$ uv run docrag review caminho/para/arquivo.py
```

O caminho pode ser absoluto ou relativo ao directorio do projeto.

O Revisor utiliza o modelo GPT-4o (modelo completo, nao o mini) para maior precisao na geracao de codigo corrigido. O Expert continua usando GPT-4o-mini.

5.2. Exemplo de revisao

Vamos revisar o proprio cli.py do projeto:

```
$ python cli.py review cli.py
```

O agente ira:

1. Ler o conteudo do arquivo
2. Executar o linter (ruff check) para identificar problemas
3. Executar o formatador (ruff format --check) para verificar formatacao
4. Analisar o codigo e sugerir melhorias
5. Retornar o CODIGO CORRIGIDO COMPLETO

Diferenca de ferramentas convencionais: o Revisor nao apenas aponta erros, mas reescreve o codigo corrigido e explica cada alteracao.

5.3. Ferramentas do revisor

O agente Revisor possui quatro ferramentas integradas:

1. read_file(caminho)

Le o conteudo do arquivo especificado. Suporta qualquer arquivo de texto (Python, Markdown, JSON, etc).

2. run_linter(caminho)

Executa ruff check no arquivo e retorna a saida do linter. Identifica erros de sintaxe, codigo morto, importacoes nao utilizadas, convencoes PEP 8, entre outros.

3. run_format_check(caminho)

Executa ruff format --check para verificar se o arquivo esta formatado corretamente segundo o estilo padrao do ruff, que segue a PEP 8 com ajustes modernos.

4. run_syntax_check(caminho)

Executa compile() no codigo Python para detectar erros de sintaxe sem executar o arquivo. Esta e a primeira linha de

defesa contra alucinacoes: se o codigo sugerido pelo modelo tiver erro de sintaxe, esta ferramenta aponta antes mesmo de voce tentar rodar.

O ruff precisa estar instalado no ambiente. Se nao estiver, instale com: uv pip install ruff

6. Playground - Interface Web

O Playground é uma interface gráfica que roda no navegador, permitindo você conversar com os agentes sem precisar digitar comandos no terminal. É ideal para quem prefere uma experiência visual interativa, similar ao ChatGPT, mas com agentes especializados na documentação do agno.

Todos os três agentes (Expert, Revisor e Tradutor) ficam disponíveis na mesma tela. Você pode alternar entre eles com um clique.

6.1. Iniciando o Playground

Com o ambiente configurado, execute:

```
$ uv run docrag playground
```

Após alguns segundos, você verá no terminal uma mensagem como:

```
Iniciando Playground do agno em http://localhost:7777
```

Abra seu navegador e acesse:

```
$ http://localhost:7777/playground
```

Opções avançadas:

```
$ uv run docrag playground --port 8080
```

```
$ uv run docrag playground --host 0.0.0.0 --port 80
```

```
$ uv run docrag playground --reload
```

--port: muda a porta (padrão 7777)

--host: expõe o servidor para outros dispositivos na rede

--reload: reinicia automaticamente ao alterar o código (desenvolvimento)

6.2. Agentes disponíveis no Playground

Agente	Modelo	Função
Expert	GPT-4o-mini	Responde perguntas sobre a documentação do agno
Revisor	GPT-4o	Le e corrige arquivos de código Python
Tradutor	GPT-4o-mini	Traduz documentos Markdown para PT-BR

6.3. Interagindo com os agentes

Ao acessar o Playground, você verá:

1. Uma lista dos agentes disponíveis no painel lateral ou no topo
2. Um campo de texto para digitar sua mensagem
3. Uma área de conversa mostrando as respostas

Como usar cada agente:

EXPERT:

- Digite sua pergunta sobre o agno em portugues ou ingles
Exemplo: "Como criar um Agent com tools?"
- O agente busca na base vetorial e responde com exemplos

REVISOR:

- Informe o caminho do arquivo que deseja revisar
Exemplo: "Revise o arquivo src/crawler.py"
- O agente le, linta, formata e retorna o codigo corrigido

TRADUTOR:

- Cole o texto em ingles que deseja traduzir
Exemplo: cole um trecho de documentacao em ingles
- O agente traduz para portugues preservando codigo

O Playground depende da mesma API Key (OPENAI_API_KEY) ja configurada no .env. Certifique-se de que ela esteja valida.

7. Cenários de Uso

7.1. Aprendendo um novo framework

Ao iniciar com o framework agno, use o Expert para tirar duvidas rapidas sem precisar ler toda a documentacao:

```
python cli.py ask "Como criar um Agent com ferramentas?"
```

O agente busca os documentos relevantes e sintetiza uma resposta objetiva com exemplos.

7.2. Code review automatizado

Antes de fazer um commit, revise o codigo com o agente Revisor:

```
python cli.py review src/crawler.py
```

Isso economiza tempo ao identificar problemas que passariam despercebidos e ja entrega o codigo corrigido.

7.3. Documentacao em PT-BR

A documentacao do agno e integralmente em ingles. O projeto docrag traduz os 1041 documentos para portugues, permitindo que desenvolvedores com menos fluencia em ingles consultem a documentacao no seu idioma nativo:

```
python cli.py ask "O que sao tools no agno?"
```

A resposta vira em portugues com exemplos em ingles (preservados para nao quebrar a sintaxe).

7.4. Pipeline CI/CD local

Integre o Revisor em um script de pre-commit:

```
python cli.py review "$@"
```

Isso garante que todo codigo commitado passe por uma revisao automatizada antes de ir para o repositorio.

8. Solucao de Problemas

Problema: OPENAI_API_KEY nao configurada

Sintoma: Erro de autenticacao ao executar ask ou review.

Solucao: Verifique se o arquivo .env existe na raiz do projeto e contem a linha OPENAI_API_KEY=sk-... .

Problema: ChromaDB vazio / nenhum documento encontrado

Sintoma: O Expert responde com conhecimento geral, sem citar a documentacao.

Solucao: Execute o pipeline completo novamente:

```
python cli.py crawl --site agno
python cli.py translate
python cli.py index
```

Problema: ruff nao encontrado

Sintoma: O Revisor retorna erro ao executar linter.

Solucao: Instale o ruff no ambiente:

```
uv pip install ruff
```

Problema: GPT-4o retorna erro de autenticacao

Sintoma: O Revisor falha ao chamar o modelo gpt-4o.

Solucao: O modelo gpt-4o requer uma conta OpenAI com acesso ao nivel adequado. Verifique se sua chave OPENAI_API_KEY tem acesso ao gpt-4o (contas de desenvolvedor tem). Se necessario, troque de volta para gpt-4o-mini no reviewer.py.

Problema: Resposta em ingles quando esperava PT-BR

Sintoma: O Expert respondeu em ingles para uma pergunta em portugues.

Solucao: Reformule a pergunta de forma clara em portugues. O agente detecta o idioma pelo texto da pergunta. Perguntas curtas ou ambíguas podem ser interpretadas como ingles.

Problema: Playground nao conecta

Sintoma: O navegador mostra 'conexao recusada' ao acessar http://localhost:7777.

Solucao: Verifique se o servidor esta rodando (olhe o terminal). Se estiver rodando em outra porta (ex: 8080), use a porta correta. Teste com: curl http://localhost:7777.

Problema: Playground inicia mas nao carrega os agentes

Sintoma: O Playground abre mas nao mostra os agentes disponiveis.

Solucao: Verifique os logs no terminal. Pode ser erro de autenticacao (OPENAI_API_KEY invalida) ou falha ao conectar no ChromaDB. Execute o pipeline de indexacao novamente se necessario.

Problema: Erro de memoria / rate limit

Sintoma: Resposta truncada ou erro HTTP 429.

Solucao: O GPT-4o-mini tem limites de uso por minuto. Aguarde alguns segundos e tente novamente.

9. Referencias

- Repositorio do projeto: <https://github.com/tsvsampaio/docrag>
- Documentacao do agno: <https://docs.agno.com>
- Documentacao do ChromaDB: <https://docs.trychroma.com>
- OpenAI API: <https://platform.openai.com>
- Ruff (linter): <https://docs.astral.sh/ruff>
- agno Playground: <https://docs.agno.com/playground>
- Tutorial gerado em 22/06/2026

docrag - Tutorial de Uso dos Agentes

Criado por tsvsampaio | github.com/tsvsampaio/docrag

Licenca: MIT